

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра "Програмна інженерія"

ЗВІТ
з лабораторної роботи №4
«Рефакторинг коду та проведення Code Review
за індустріальними стандартами»
з дисципліни
«Основи програмної інженерії»

Виконала
ст. гр. ПЗП-25-4

Бондаренко Тетяна
Олександрівна

Прийняв:
В. І. Каук,
В.О. Гребенюк

Харків 2026

ЛАБОРАТОРНА РОБОТА 4 РЕФАКТОРИНГ КОДУ ТА ПРОВЕДЕННЯ CODE REVIEW ЗА ІНДУСТРІАЛЬНИМИ СТАНДАРТАМИ

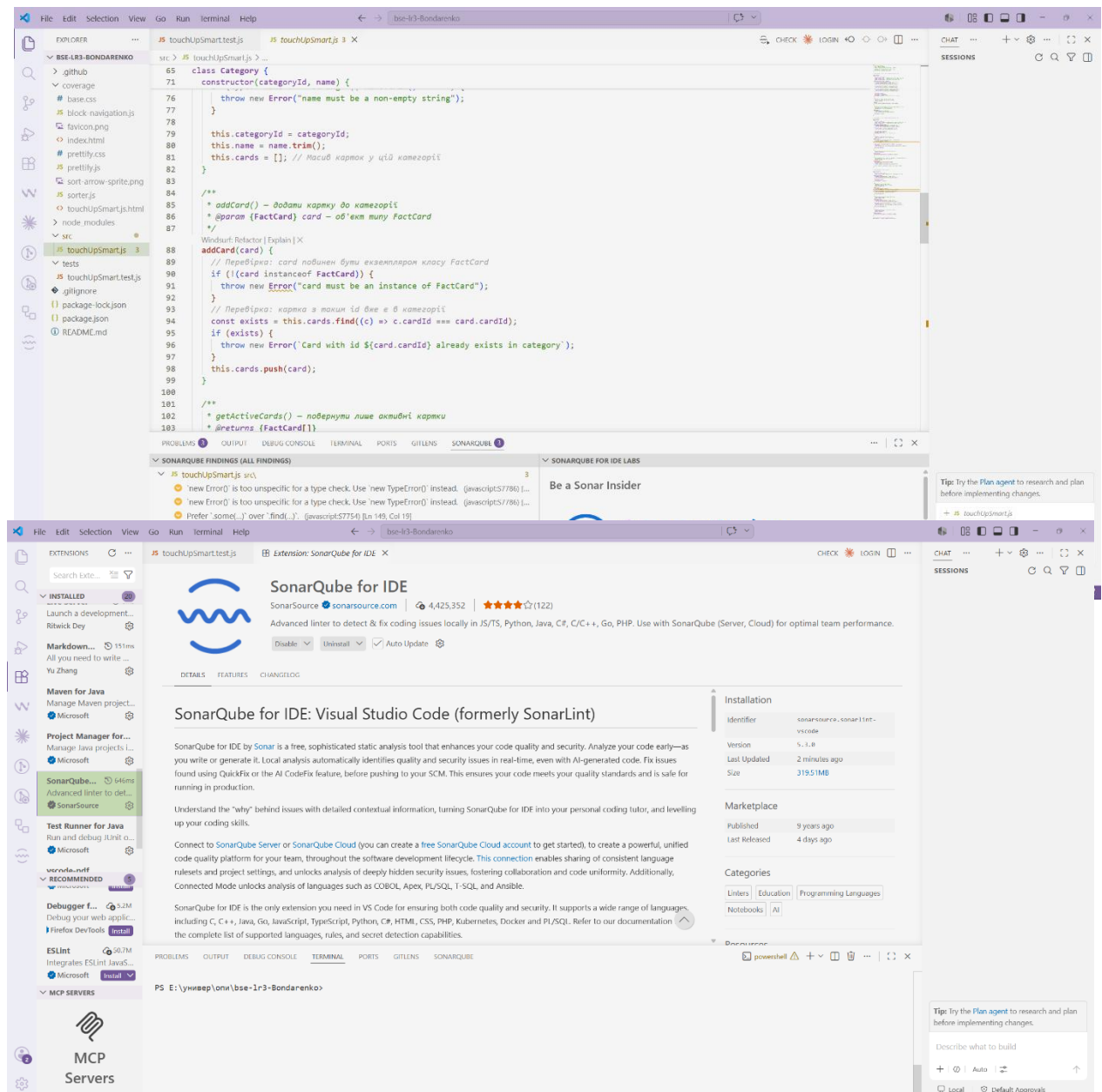
4.1 Мета роботи

Набуття практичних навичок із проведення Code Review за індустріальними стандартами з використанням GitHub Pull Requests. Оволодіння методами ідентифікації та класифікації типових проблем якості коду (code smells). Отримання досвіду застосування рефакторинг-операцій із верифікацією збереження поведінки через регресійне тестування.

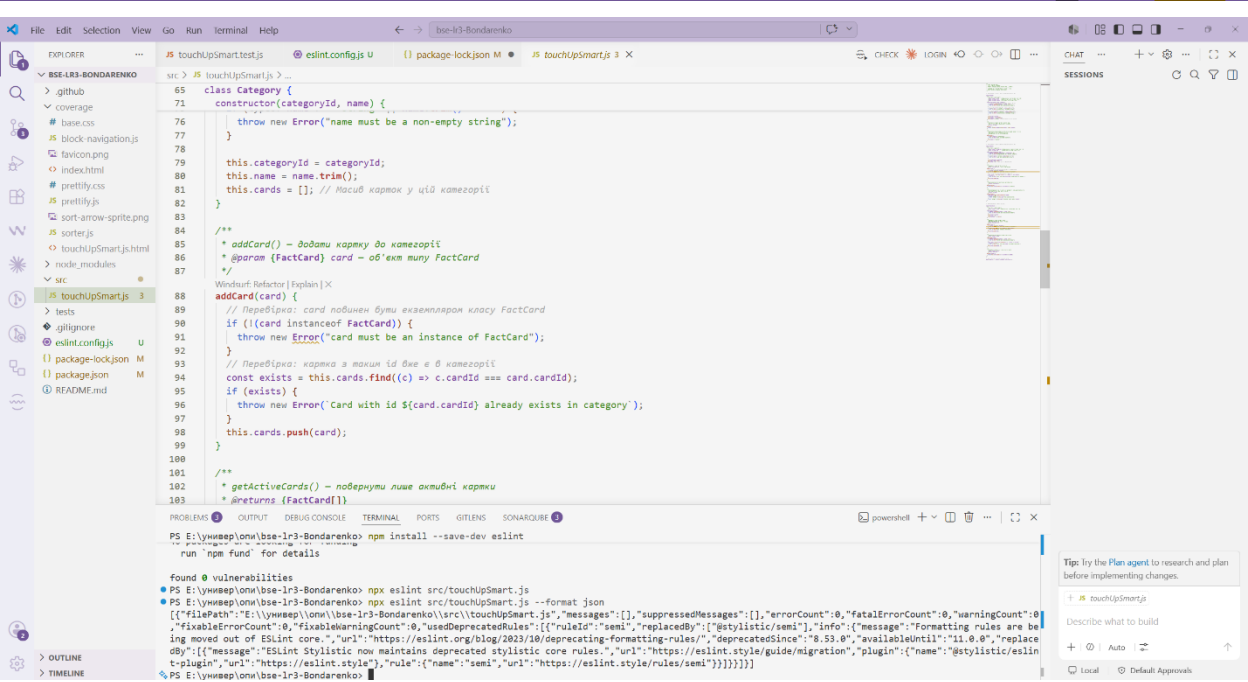
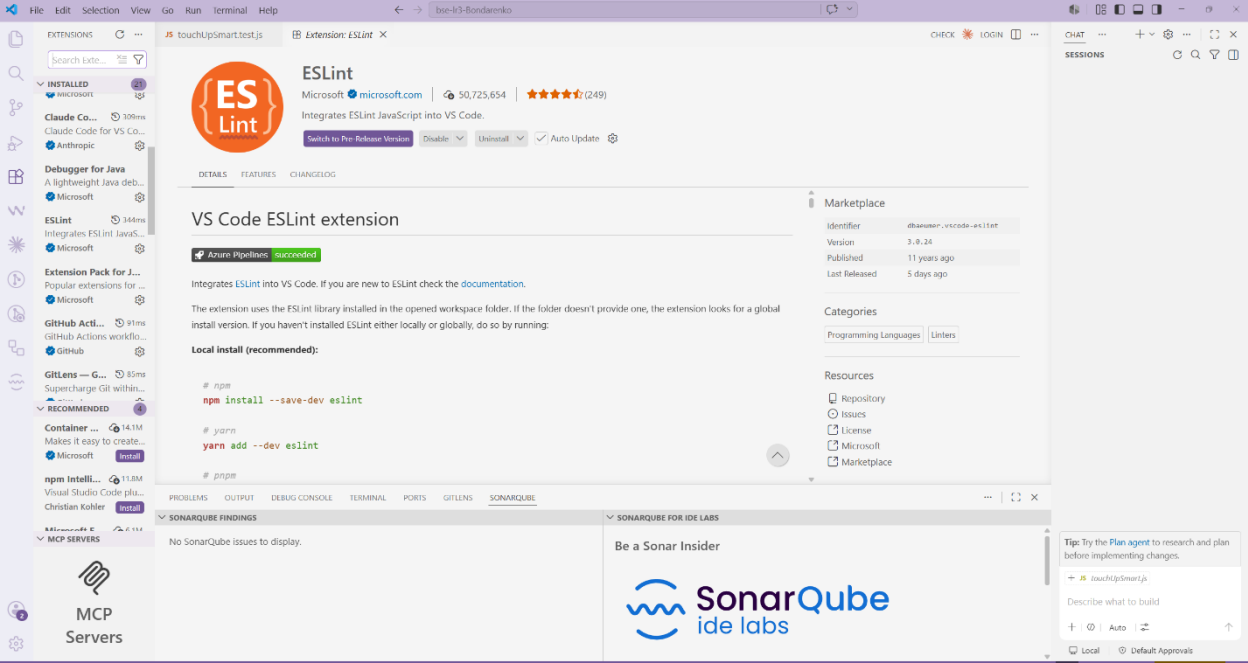
Етап 1. Підготовка

<https://github.com/tetianabondarenko3-oss/bse-lr3-Bondarenko.git>

1. Встановити SonarQube for IDE (SonarLint) у VS Code. Запустити аналіз свого коду та зафіксувати початкову кількість issues



До: Sonar Qube, 3 issues (скріншот «до»).



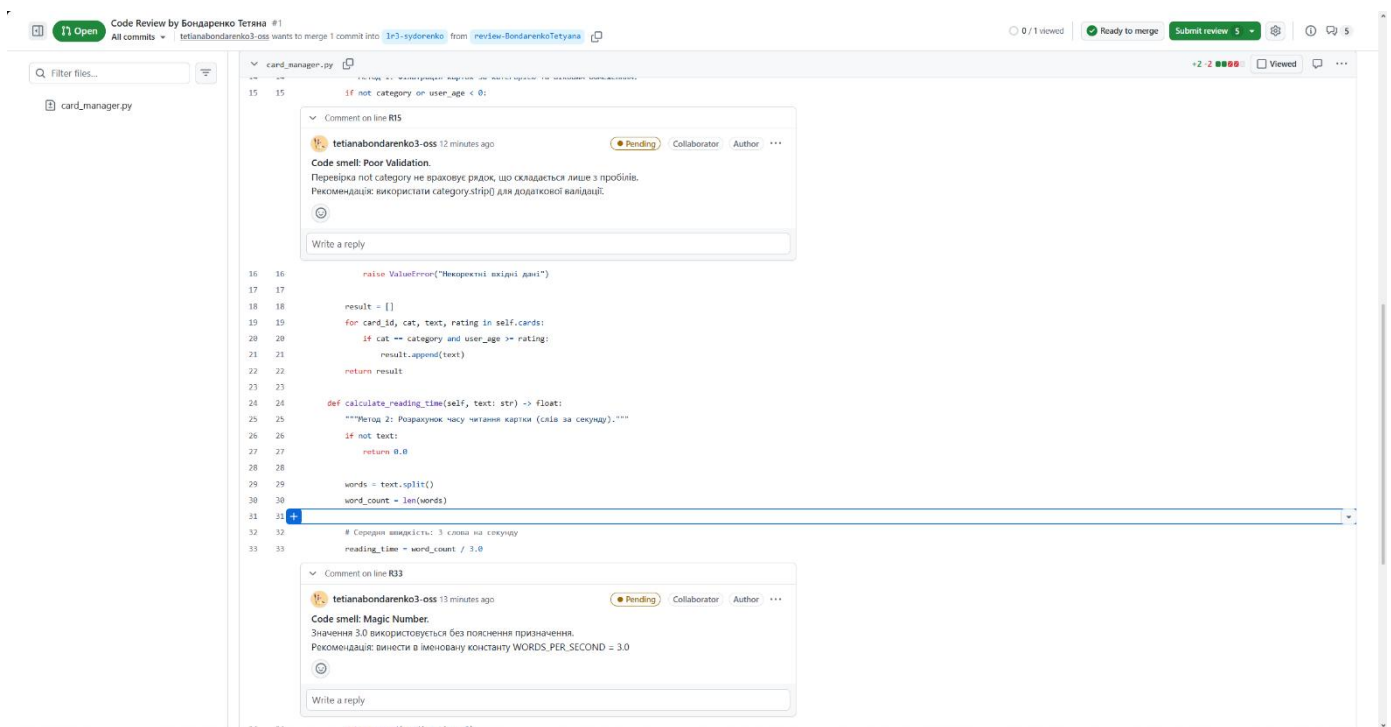
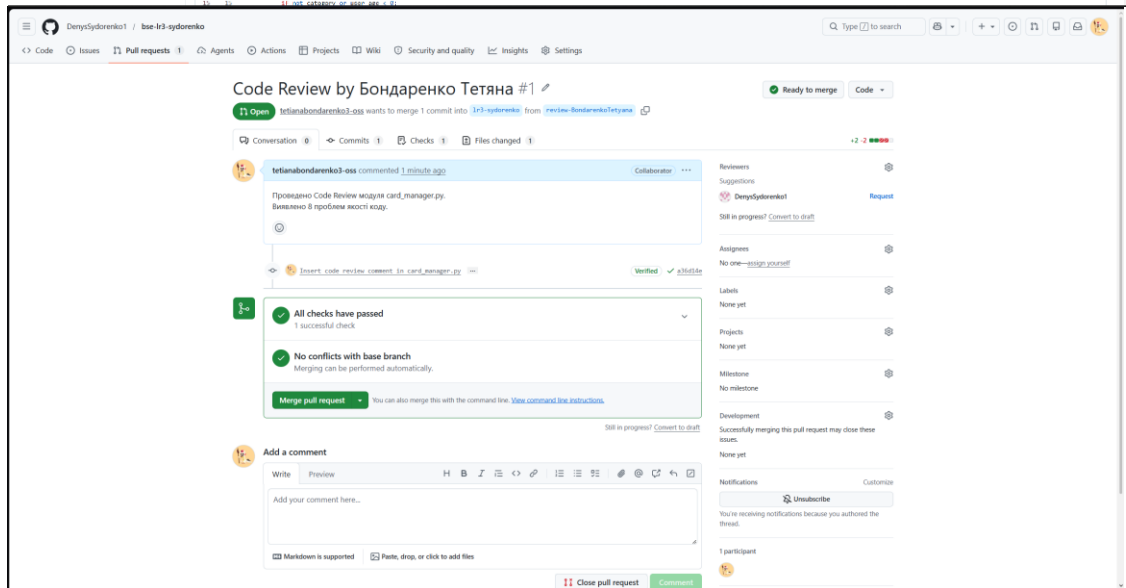
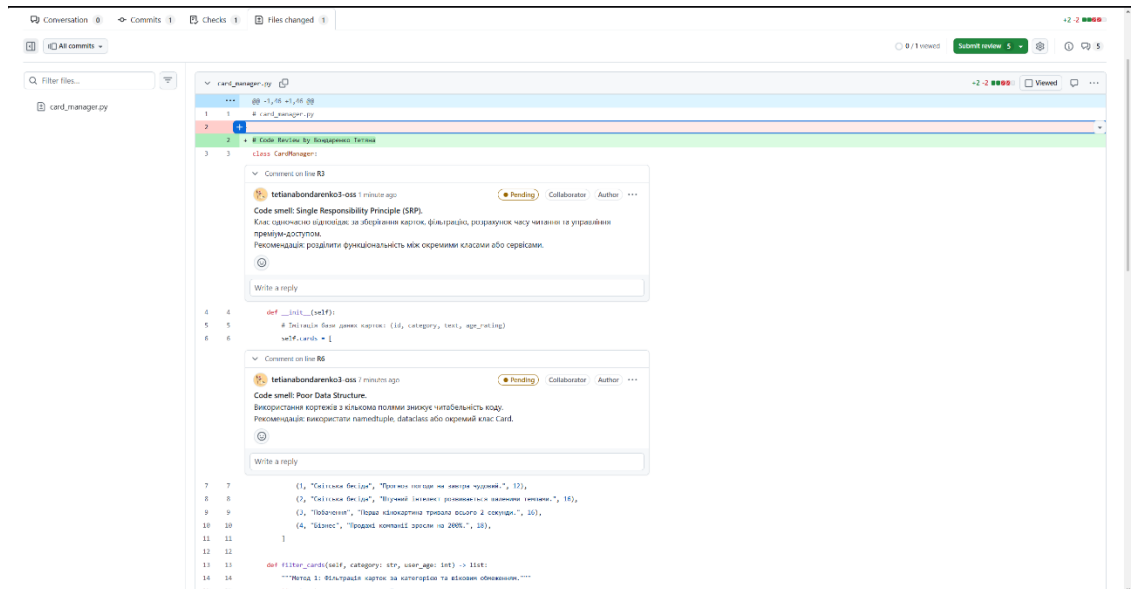
Инструмент	До рефакторингу
ESLint warnings	0
ESLint errors	0

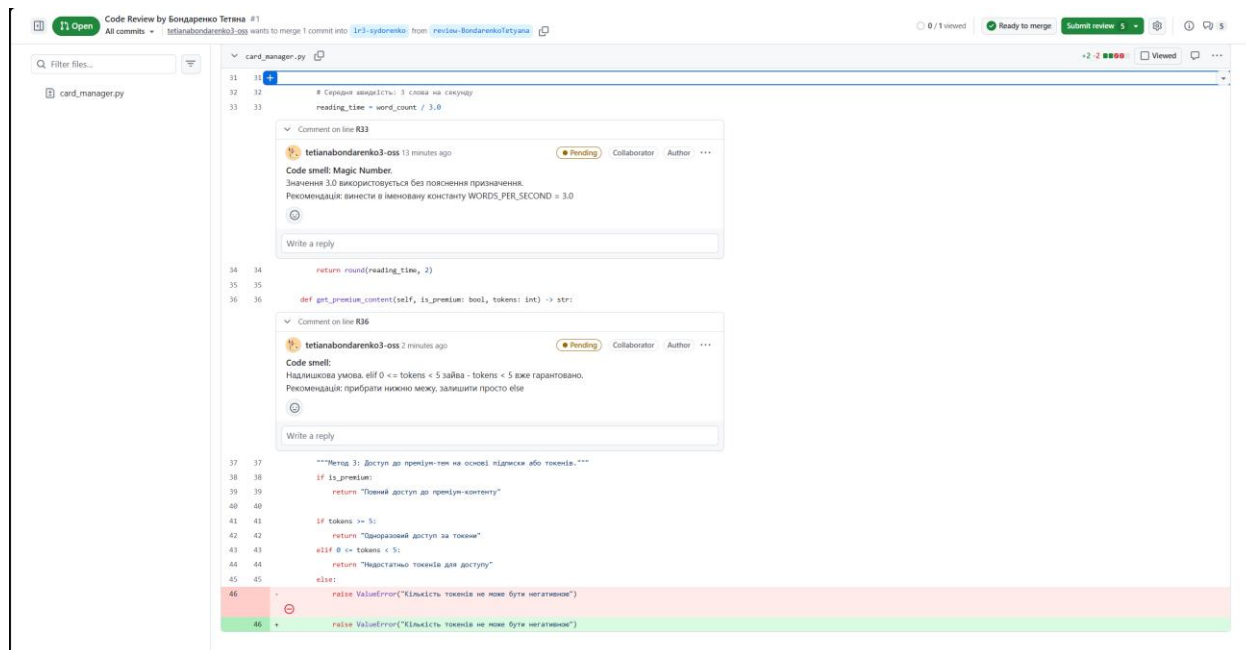
Етап 2. Code Review

2. Результати Code Review коду Дениса Сидоренка

У рамках взаємного Code Review було проаналізовано модуль card_manager.py Дениса Сидоренка. Виявлено 5 проблем якості коду, задокументованих як inline-коментарі у Pull Request на GitHub. Таблиця нижче містить зведені результати.

№	Рядок коду	Проблема	Категорія	Рекомендація
1	Клас CardManager (рядок 3)	Клас відповідає за зберігання, фільтрацію, розрахунок і преміум-доступ одночасно	Порушення SRP	Розділити на окремі класи: CardRepository, CardFilter, ReadingTimeCalculator, PremiumAccessService
2	self.cards = [...] (рядок 6)	Дані карток зберігаються як кортежі - знижує читабельність	Погана структура даних	Використати @dataclass або namedtuple Card(id, category, text, age_rating)
3	if not category (рядок 15)	Перевірка не відловлює рядки з пробілами (наприклад, ' ')	Погана валідація	Замінити на if not category.strip() або category.strip() == "
4	word_count / 3.0 (рядок 33)	Магічне число 3.0 без пояснення - незрозуміло, що означає	Магічне число	Винести в константу: WORDS_PER_SECOND = 3.0 на рівні модуля
5	elif 0 <= tokens < 5 (рядок 41)	Умова 0 <= tokens надлишкова: якщо tokens >= 5 не виконалось, tokens < 5 вже гарантовано	Надлишкова умова / мертвий код	Прибрати нижню межу: замінити elif 0 <= tokens < 5 на else, а окремий else з ValueError залишити





3. Посилання на Pull Request з коментарями Code Review

Pull Request із коментарями Code Review розміщено у репозиторії одногрупника на GitHub:

<https://github.com/DenysSydorenko1/bse-lr3-sydorenko/pull/1>

Гілка містить усі inline-коментарі до файлу `src/card_manager.py` з описом кожної проблеми, її категорії та рекомендації щодо виправлення.

Етап 3. Рефакторинг власного коду

За результатами Code Review від Гліба Байстова для файлу `src/touchUpSmart.js` виконано 4 рефакторинг-операції. Кожна операція задокументована у форматі БУЛО → СТАЛО → ЧОМУ.

Звіти регресійного тестування: скріншоти результату Jest - усі тести green після рефакторингу. А також зведена таблиця.

Операція 1 - Replace Magic Strings with Constants

Проблема (БУЛО)

Статуси карток "active" та "inactive" були захардкожені як рядкові літерали безпосередньо в конструкторі та методах класу FactCard. Це порушення патерну Primitive Obsession: будь-яка опечатка у рядку не буде виявлена ні лінером, ні IDE.

```
// FactCard - constructor
```

```

this.status = "active";          // magic string

// FactCard - deactivate()
if (this.status === "inactive") { // magic string
  throw new Error("Card is already inactive");
}
this.status = "inactive";        // magic string

// Category - getActiveCards()
return this.cards.filter((c) => c.status === "active"); // magic string

```

Виправлення (СТАЛО)

Введено об'єкт-константу CARD_STATUS на рівні модуля. Усі рядкові літерали замінено на звернення до цієї константи.

```

const CARD_STATUS = {
  ACTIVE: "active",
  INACTIVE: "inactive",
};

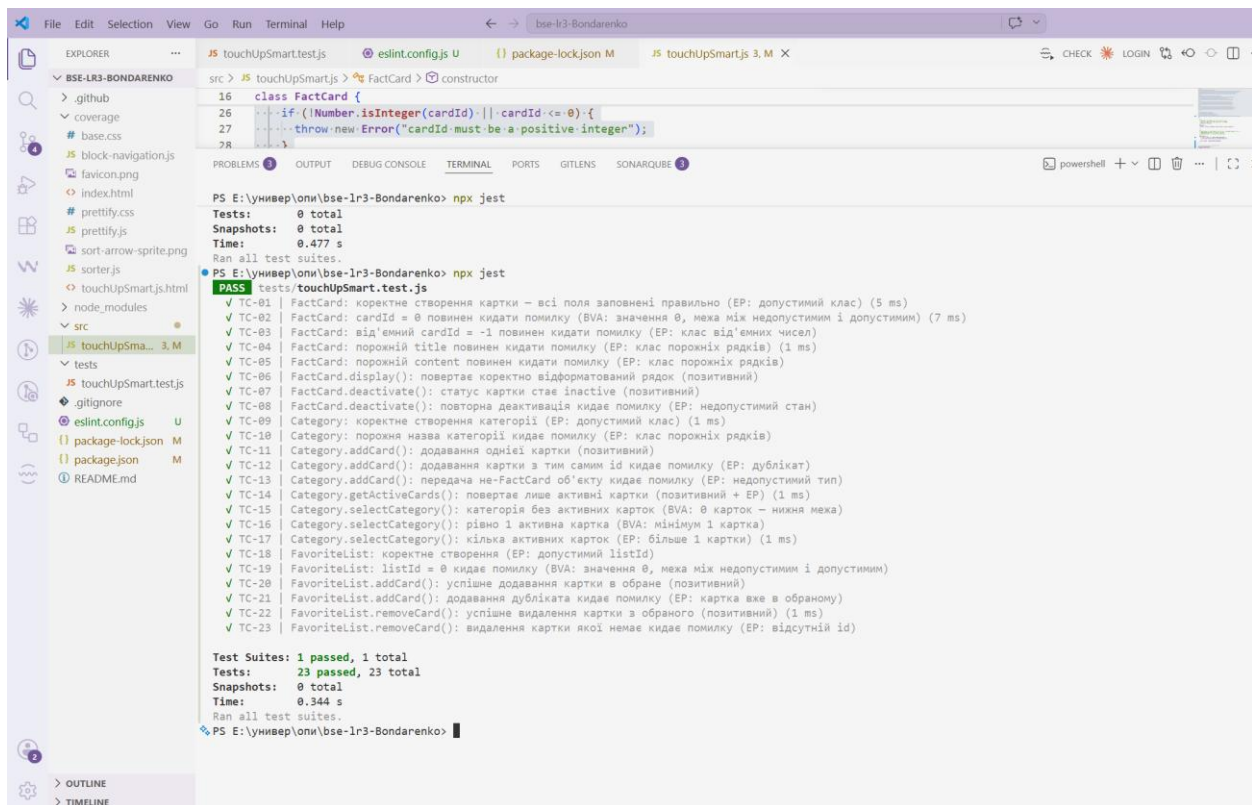
// FactCard - constructor
this.status = CARD_STATUS.ACTIVE;

// FactCard - deactivate()
if (this.status === CARD_STATUS.INACTIVE) {
  throw new Error("Card is already inactive");
}
this.status = CARD_STATUS.INACTIVE;

// Category - getActiveCards()
return this.cards.filter((c) => c.status === CARD_STATUS.ACTIVE);

```

ЧОМУ: Опечатка у рядковому літералі не спричиняє помилки виконання, тому дефект важко відловити. Константа CARD_STATUS усуває цей ризик - IDE підказує допустимі значення, а зміна статусу потребує редагування лише в одному місці.



Операція 2 - Extract Validation Method (DRY)

Проблема (БУЛО)

Однакова перевірка «позитивне ціле число» дублювалася у 4 місцях: конструкторах FactCard, Category, FavoriteList та методі removeCard. Порушення принципу DRY (Don't Repeat Yourself): при зміні правила валідації треба було б правити 4 окремих місця.

```
// У конструкторі FactCard:  
if (!Number.isInteger(cardId) || cardId <= 0) {  
  throw new Error("cardId must be a positive integer");  
}
```

```
// У конструкторі Category (та сама логіка):  
if (!Number.isInteger(categoryId) || categoryId <= 0) {  
  throw new Error("categoryId must be a positive integer");  
}
```

```
// У конструкторі FavoriteList (та сама логіка):  
if (!Number.isInteger(listId) || listId <= 0) {  
  throw new Error("listId must be a positive integer");  
}
```

```
// У методі removeCard (та сама логіка):  
if (!Number.isInteger(cardId) || cardId <= 0) {  
  throw new Error("cardId must be a positive integer");  
}
```

Виправлення (СТАЛО)

Логіку перевірки винесено в окрему допоміжну функцію `validatePositiveInteger(value, paramName)`, яка використовується скрізь.

```
function validatePositiveInteger(value, paramName) {
  if (!Number.isInteger(value) || value <= 0) {
    throw new Error(`${paramName} must be a positive integer`);
  }
}
```

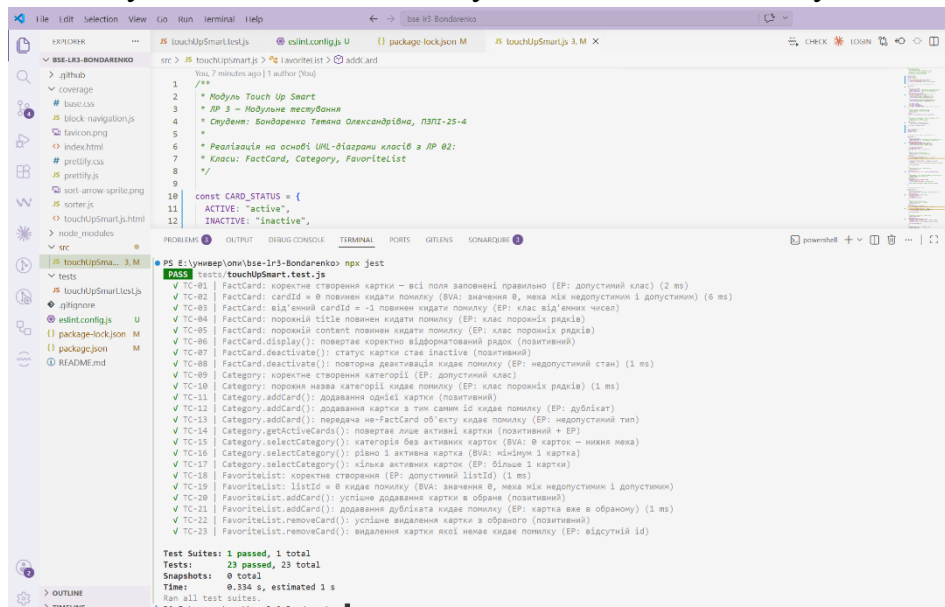
```
// FactCard - constructor:
validatePositiveInteger(cardId, "cardId");
```

```
// Category - constructor:
validatePositiveInteger(categoryId, "categoryId");
```

```
// FavoriteList - constructor:
validatePositiveInteger(listId, "listId");
```

```
// FavoriteList - removeCard:
validatePositiveInteger(cardId, "cardId");
```

ЧОМУ: Усунено 4 дублікати. Тепер правило валідації живе в одному місці. Параметр `paramName` забезпечує інформативне повідомлення про помилку для кожного виклику без додаткового коду.



Операція 3 - Replace `find()` with `some()` Проблема (БУЛО)

У методі `Category.addCard` для перевірки наявності картки використовувався `.find()`, який повертає знайдений об'єкт або `undefined`.

Однак сам знайдений об'єкт далі не використовувався - перевірявся лише факт існування через truthy-значення.

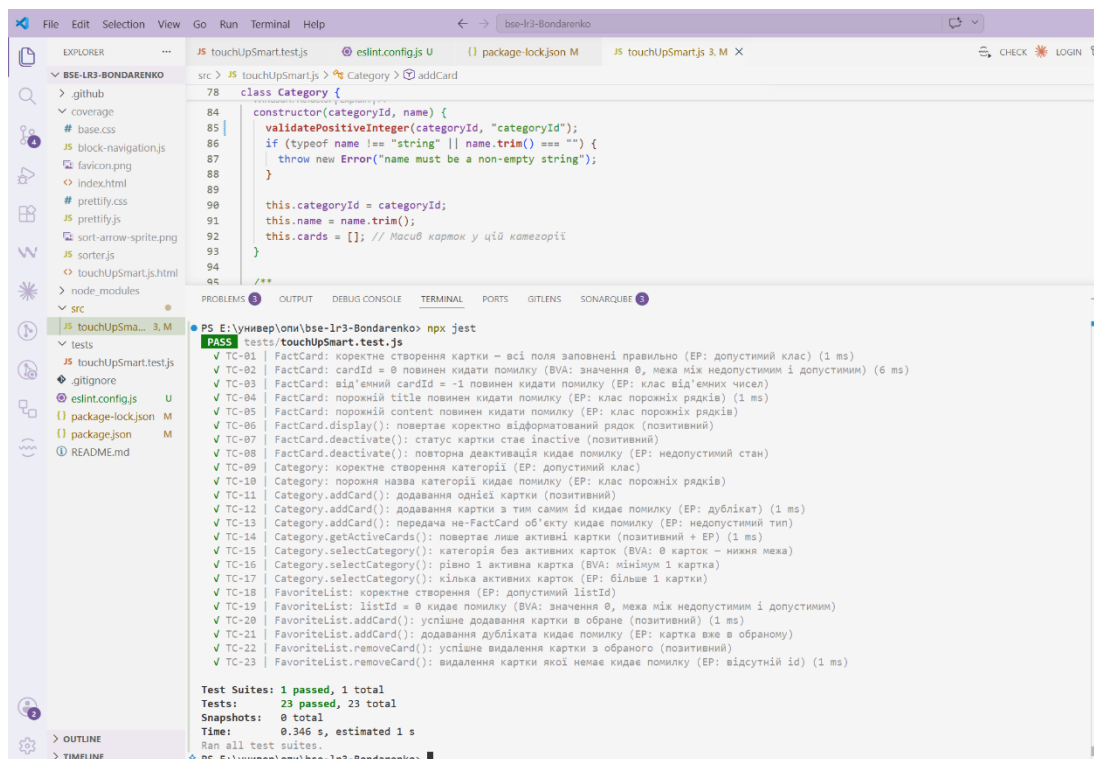
```
// Category - addCard():
const exists = this.cards.find((c) => c.cardId === card.cardId);
if (exists) {
  throw new Error(`Card with id ${card.cardId} already exists in category`);
}
```

Виправлення (СТАЛО)

Замінено на `.some()`, який семантично точно відображає намір: «чи існує хоча б один елемент, що відповідає умові».

```
// Category - addCard():
if (this.cards.some((c) => c.cardId === card.cardId)) {
  throw new Error(`Card with id ${card.cardId} already exists in category`);
}
```

ЧОМУ: `.some()` повертає boolean і явно виражає намір - потрібен лише факт існування. `.find()` повертає об'єкт, що вводить читача в оману: «де далі використовується `exists?`». Крім того, `.some()` трохи ефективніше - зупиняється одразу після першого збігу і не продовжує обхід масиву.



The screenshot shows a VS Code editor with a file explorer on the left showing the project structure. The main editor displays the `Category` class in `src > JS touchUpSmart.js`. The class has a `constructor` and a `validatePositiveInteger` method. The `addCard` method is highlighted. The terminal window at the bottom shows the command `PS E:\univer\oni\bse-lr3-Bondarenko> npx jest` and the output of the Jest test runner, indicating that all tests passed.

```
PS E:\univer\oni\bse-lr3-Bondarenko> npx jest
PASS tests\touchUpSmart.test.js
  ✓ TC-01 | FactCard: коректне створення картки – всі поля заповнені правильно (EP: допустимий клас) (1 ms)
  ✓ TC-02 | FactCard: cardId = 0 повинен кидати помилку (BVA: значення 0, межа між недопустимим і допустимим) (6 ms)
  ✓ TC-03 | FactCard: sid'єний cardId = -1 повинен кидати помилку (EP: клас від'єнних чисел)
  ✓ TC-04 | FactCard: порожній title повинен кидати помилку (EP: клас порожніх рядків) (1 ms)
  ✓ TC-05 | FactCard: порожній content повинен кидати помилку (EP: клас порожніх рядків)
  ✓ TC-06 | FactCard.display(): повертає коректно відформатований рядок (позитивний)
  ✓ TC-07 | FactCard.deactivate(): статус картки стає inactive (позитивний)
  ✓ TC-08 | FactCard.deactivate(): повторна деактивація кидє помилку (EP: недопустимий стан)
  ✓ TC-09 | Category: коректне створення категорії (EP: допустимий клас)
  ✓ TC-10 | Category: порожня назва категорії кидє помилку (EP: клас порожніх рядків)
  ✓ TC-11 | Category.addCard(): додавання однієї картки (позитивний)
  ✓ TC-12 | Category.addCard(): додавання картки з тин самим id кидє помилку (EP: дублікат) (1 ms)
  ✓ TC-13 | Category.addCard(): передача не-FactCard об'єкту кидє помилку (EP: недопустимий тип)
  ✓ TC-14 | Category.getActiveCards(): повертає лише активні картки (позитивний * EP) (1 ms)
  ✓ TC-15 | Category.selectCategory(): категорія без активних карток (BVA: 0 карток – нижня межа)
  ✓ TC-16 | Category.selectCategory(): рівно 1 активна картка (BVA: мінімум 1 картка)
  ✓ TC-17 | Category.selectCategory(): кілька активних карток (EP: більше 1 картки)
  ✓ TC-18 | FavoriteList: коректне створення (EP: допустимий listId)
  ✓ TC-19 | FavoriteList: listId = 0 кидє помилку (BVA: значення 0, межа між недопустимим і допустимим)
  ✓ TC-20 | FavoriteList.addCard(): успішне додавання картки в обране (позитивний) (1 ms)
  ✓ TC-21 | FavoriteList.addCard(): додавання дублюєтата кидє помилку (EP: картка вже в обраному)
  ✓ TC-22 | FavoriteList.removeCard(): успішне видалення картки з обраного (позитивний)
  ✓ TC-23 | FavoriteList.removeCard(): видалення картки якої немає кидє помилку (EP: відсутній id) (1 ms)

Test Suites: 1 passed, 1 total
Tests: 23 passed, 23 total
Snapshots: 0 total
Time: 0.346 s, estimated 1 s
Run all test suites.
```

Операція 4 - Reuse hasCard() замість повторної логіки find() Проблема (БУЛО)

У методі `FavoriteList.addCard` логіка перевірки наявності картки дублювала вже реалізований у цьому ж класі метод `hasCard(cardId)`. Це порушення DRY всередині одного класу.

```
// FavoriteList - addCard():
if (this.cards.find((c) => c.cardId === card.cardId)) { // дублювання!
  throw new Error("Card already in favorites");
}
```

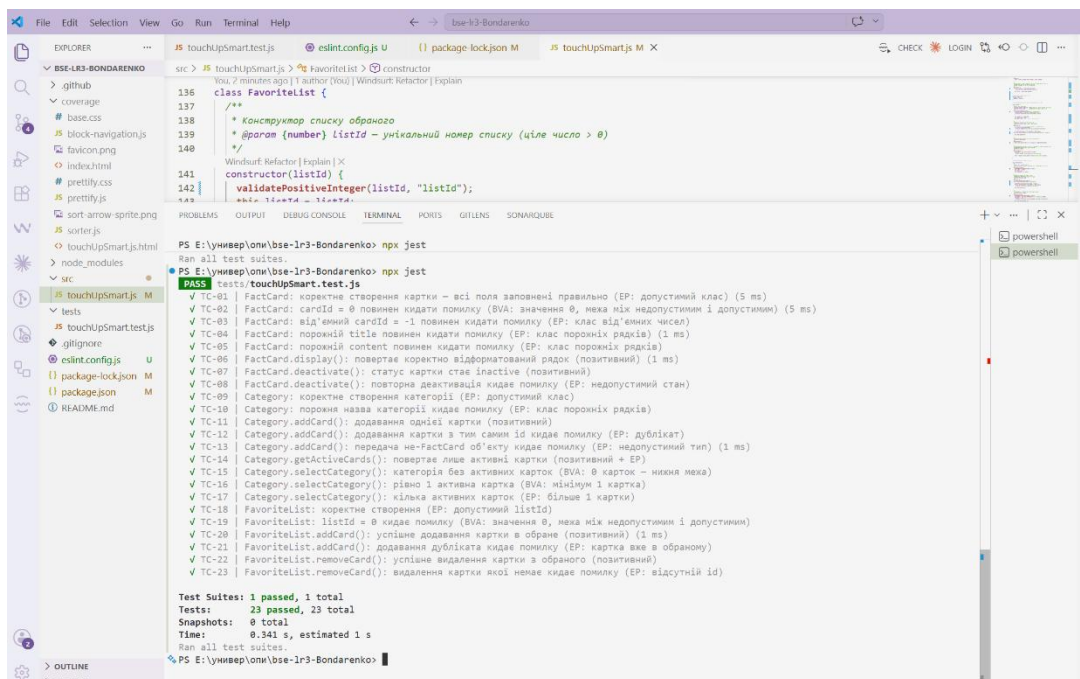
```
// FavoriteList - hasCard() - той самий метод вже є в класі:
hasCard(cardId) {
  return this.cards.some((c) => c.cardId === cardId);
}
```

Виправлення (СТАЛО)

Виклик `find()` замінено на перевикористання існуючого методу `hasCard()`.

```
// FavoriteList - addCard():
if (this.hasCard(card.cardId)) {
  throw new Error("Card already in favorites");
}
```

ЧОМУ: Якщо логіка перевірки `hasCard()` зміниться (наприклад, додасться перевірка статусу), достатньо оновити лише один метод. `addCard()` стає коротшим і читабельнішим - намір «якщо картка вже є» виражається через ім'я методу, а не через деталі реалізації.



Зведена таблиця рефакторинг-операцій (БУЛО → СТАЛО)

№	Операція	БУЛО	СТАЛО
1	Replace Magic Strings with Constants	<code>this.status = "active" if (this.status === "inactive")</code>	<code>const CARD_STATUS = { ACTIVE: "active", INACTIVE: "inactive" } this.status = CARD_STATUS.ACTIVE if (this.status === CARD_STATUS.INACTIVE)</code>
2	Extract Validation Method (DRY)	<code>// FactCard: if (!Number.isInteger(cardId) cardId <= 0) { throw new Error("cardId must be a positive integer"); } // Category (те came): if (!Number.isInteger(categoryId) categoryId <= 0) { throw new Error(...); } // FavoriteList (те came): if (!Number.isInteger(listId) ...</code>	<code>function validatePositiveInteger(value, paramName) { if (!Number.isInteger(value) value <= 0) { throw new Error(` \${paramName} must be a positive integer`); } } // Скрізь замість дублювання: validatePositiveInteger(cardId, "cardId"); validatePositiveInteger(categoryId, "categoryId"); validatePositiveInteger(listId, "listId"); validatePositiveInteger(cardId, "cardId"); // removeCard</code>
3	Replace find() with some()	<code>// Category.addCard: const exists = this.cards.find((c) => c.cardId === card.cardId); if (exists) { throw ... }</code>	<code>// Category.addCard: if (this.cards.some((c) => c.cardId === card.cardId)) { throw ... }</code>
4	Reuse hasCard() замість повторно її логіки find()	<code>// FavoriteList.addCard: if (this.cards.find((c) => c.cardId === card.cardId)) { throw new Error("Card already in favorites"); }</code>	<code>// FavoriteList.addCard: if (this.hasCard(card.cardId)) { throw new Error("Card already in favorites"); }</code>

Обґрунтування рефакторинг-операцій (ЧОМУ)

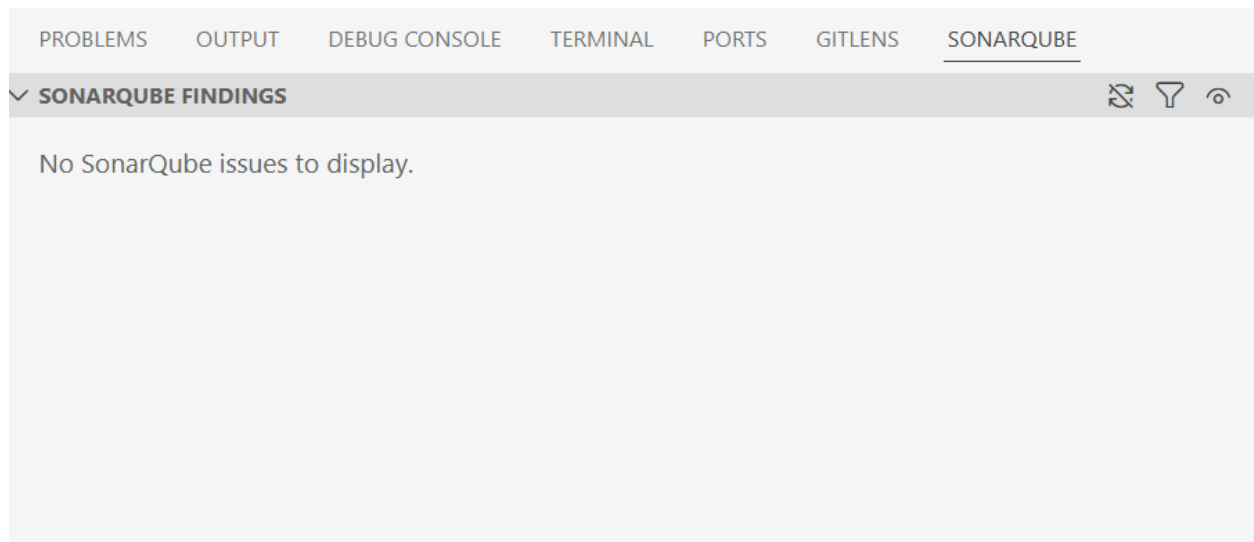
№	Операція	ЧОМУ (обґрунтування)
1	Replace Magic Strings with Constants	Захардкожені рядки "active"/"inactive" легко зіпсувати опечаткою, а IDE не підкаже помилку. Константа CARD_STATUS дозволяє змінити значення в одному місці та отримати автодоповнення.
2	Extract Validation Method (DRY)	Однакова перевірка дублювалась у 4 місцях (3 конструктори + removeCard). При зміні правила довелося б правити скрізь. Єдина функція validatePositiveInteger() усуває дублювання і спрощує тестування.
3	Replace find() with some()	find() повертає об'єкт або undefined, але далі він ніде не використовується - перевіряється лише факт існування. some() семантично точніше (повертає boolean) і дещо ефективніше (зупиняється одразу після першого збігу).
4	Reuse hasCard() замість повторної логіки find()	У FavoriteList вже був метод hasCard(), але addCard() дублював його логіку через find(). Перевикористання hasCard() дотримується DRY: якщо логіка перевірки зміниться, достатньо оновити лише один метод.

Етап 4. Верифікація та звітність.

Повторно запустити SonarLint та лінтер. Зафіксувати кількість issues «після»

ESLint issues до та після рефакторингу: 0

```
Ran all test suites.  
● PS E:\универ\опи\bse-1r3-Bondarenko> npx eslint src/touchUpSmart.js  
❖ PS E:\универ\опи\bse-1r3-Bondarenko> █
```



До рефакторингу: SonarQube 3 issues
Після рефакторингу: 0 issues

Запустити обчислення метрик складності lizard. Порівняти з початковими значеннями.

До:

```
=====
NLOC    CCN    token  PARAM  length  location
-----
15      7     112    3      19     constructor@20-38@src_original/touchUpSmart (1).js
3       1     20     0       3     display@45-47@src_original/touchUpSmart (1).js
6       2     27     0       6     deactivate@54-59@src_original/touchUpSmart (1).js
11      5     77     2      12     constructor@71-82@src_original/touchUpSmart (1).js
1       1     8      0       1     (anonymous)@94-94@src_original/touchUpSmart (1).js
10      3     66     1      12     addCard@88-99@src_original/touchUpSmart (1).js
1       1     6      0       1     (anonymous)@106-106@src_original/touchUpSmart (1).js
3       1     17     0       3     getActiveCards@105-107@src_original/touchUpSmart (1).js
7       2     44     0       7     selectCategory@114-120@src_original/touchUpSmart (1).js
8       3     50     1       8     constructor@131-138@src_original/touchUpSmart (1).js
1       1     8      0       1     (anonymous)@149-149@src_original/touchUpSmart (1).js
9       3     55     1       9     addCard@145-153@src_original/touchUpSmart (1).js
1       1     6      0       1     (anonymous)@163-163@src_original/touchUpSmart (1).js
10      4     76     1      10     removeCard@159-168@src_original/touchUpSmart (1).js
1       1     6      0       1     (anonymous)@176-176@src_original/touchUpSmart (1).js
3       1     18     1       3     hasCard@175-177@src_original/touchUpSmart (1).js
1 file analyzed.
=====
NLOC    Avg.NLOC  AvgCCN  Avg.token  function_cnt  file
-----
92      5.6      2.3     37.2       16            src_original/touchUpSmart (1).js
=====
No thresholds exceeded (cyclomatic_complexity > 15 or length > 1000 or nloc > 1000000 or parameter_count > 100)
=====
Total nloc  Avg.NLOC  AvgCCN  Avg.token  Fun Cnt  Warning cnt  Fun Rt  nloc Rt
-----
92      5.6      2.3     37.2       16         0         0.00   0.00
=====
```

Після:

```
PS E:\универ\опи\bse-lr3-Bondarenko> python -m lizard src/
=====
NLOC   CCN   token  PARAM  length  location
-----
5       3     35     2       5   validatePositiveInteger@14-18@src/touchUpSmart.js
13      5     98     3       17  constructor@29-45@src/touchUpSmart.js
3       1     20     0        3   display@52-54@src/touchUpSmart.js
6       2     31     0        6   deactivate@61-66@src/touchUpSmart.js
9       3     61     2       10  constructor@84-93@src/touchUpSmart.js
1       1      8     0        1   (anonymous)@105-105@src/touchUpSmart.js
9       3     61     1       11  addCard@99-109@src/touchUpSmart.js
1       1      8     0        1   (anonymous)@116-116@src/touchUpSmart.js
3       1     17     0        3   getActiveCards@115-117@src/touchUpSmart.js
7       2     44     0        7   selectCategory@124-130@src/touchUpSmart.js
6       1     34     1        6   constructor@141-146@src/touchUpSmart.js
1       1      8     0        1   (anonymous)@158-158@src/touchUpSmart.js
10      3     66     1       10  addCard@153-162@src/touchUpSmart.js
1       1      6     0        1   (anonymous)@170-170@src/touchUpSmart.js
8       2     60     1        8   removeCard@168-175@src/touchUpSmart.js
1       1      6     0        1   (anonymous)@183-183@src/touchUpSmart.js
3       1     18     1        3   hasCard@182-184@src/touchUpSmart.js

1 file analyzed.
=====
NLOC   Avg.NLOC  AvgCCN  Avg.token  function_cnt  file
-----
97      5.1      1.9     34.2       17   src/touchUpSmart.js

=====
No thresholds exceeded (cyclomatic_complexity > 15 or length > 1000 or nloc > 1000000 or parameter_count > 100)
=====
Total nloc   Avg.NLOC  AvgCCN  Avg.token  Fun Cnt  Warning cnt  Fun Rt  nloc Rt
-----
97      5.1      1.9     34.2       17         0      0.00  0.00
```

Метрика	ДО	ПІСЛЯ
SonarLint issues	3	0
ESLint warnings	0	0
AvgCCN (середня складність)	2.3	1.9
CCN max (FactCard constructor)	7	5
CCN max (Category constructor)	5	3
CCN max (removeCard)	4	2
NLOC (рядків коду)	92	97
function cnt	16	17
Тести (pass / total)	23/23	23/23

FactCard constructor: 7 → 5 - три if з подвійними умовами через ||. Після рефакторингу перевірка cardId пішла у validatePositiveInteger, залишились тільки title і content.

Category constructor: 5 → 3 - те саме, перевірка categoryId винесена у validatePositiveInteger.

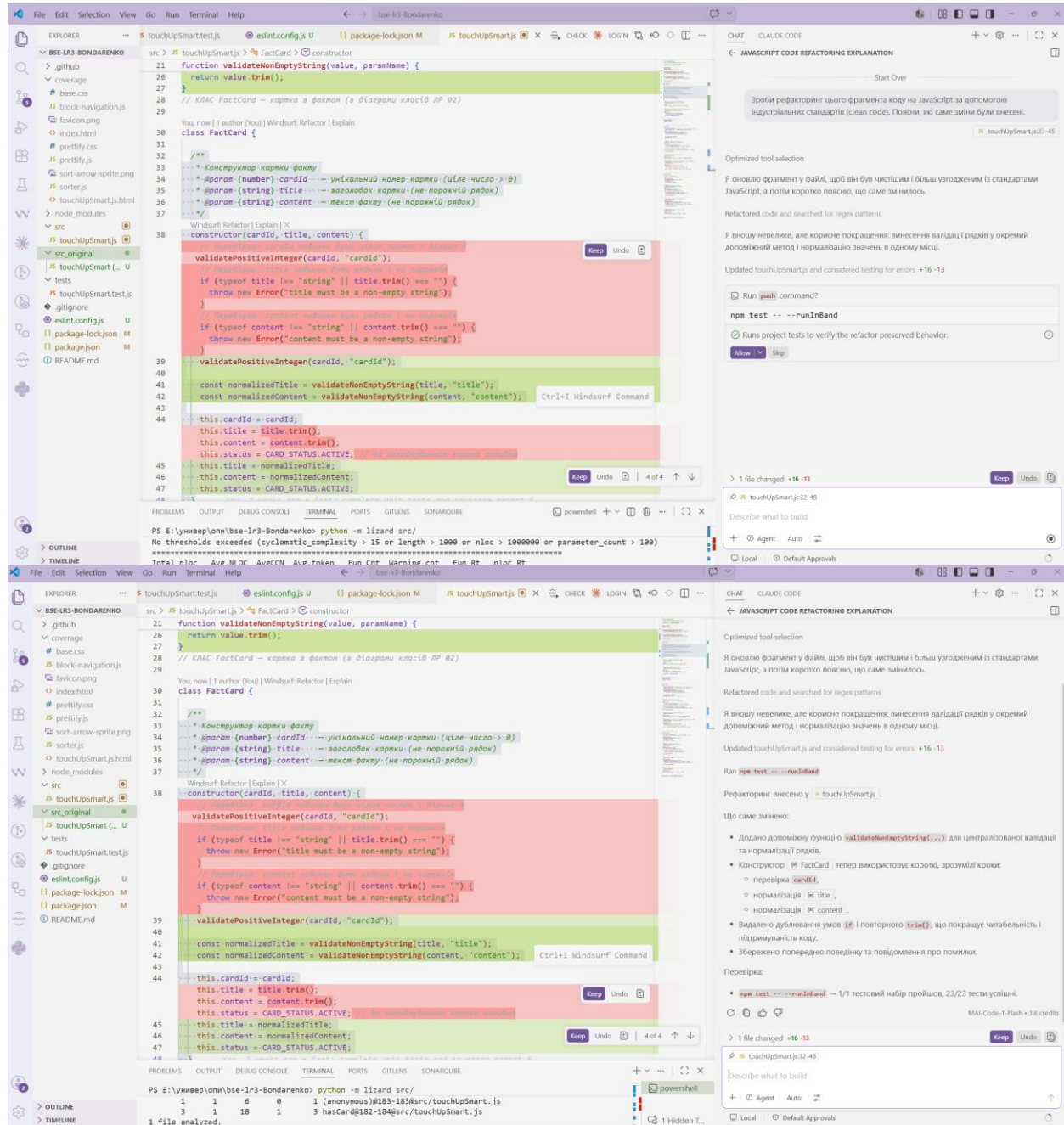
removeCard: 4 → 2 - перевірка cardId винесена у validatePositiveInteger, залишився тільки if (index === -1).

addCard FavoriteList: 3 → 3 - не змінилась, бо hasCard() просто замінив find() без зміни кількості розгалужень.

function_cnt 16 → 17 - додалась нова функція validatePositiveInteger.

Бонусне завдання: використати **GitHub Copilot Chat** для отримання пояснення підозрілого фрагмента коду (Explain this code) або пропозиції рефакторингу (Suggest refactoring). Результат включити у звіт.

Suggest refactoring:



Підсумкова рефлексія

Найціннішим досвідом під час виконання лабораторної роботи стало усвідомлення того, що Code Review - конструктивний діалог між розробниками про якість коду. Особливо корисним виявився практичний досвід аналізу чужого коду: переглядаючи модуль Дениса, значно легше помітити такі проблеми, як порушення SRP та магічні числа, ніж у власному коді.

Рефакторинг за результатами Code Review показав, наскільки важливим є регресійне тестування: після кожної зміни запуск тестів давав впевненість, що поведінка модуля не порушена, навіть коли структура коду суттєво змінювалася.

Взаємний формат Code Review (кожен є і автором, і рецензентом) максимально наближає навчальний процес до реальної командної розробки на гітхабі та формує звичку критично оцінювати власні рішення.